

Master's Degree in Artificial Intelligence

AI & Cloud Project

**Distributed Community
Detection with Spark
and GraphFrames**

Academic Year: 2025/2026

Enrico Loda - VR522872
Giacomo Lucato - VR522918

University of Verona
Department of Computer Science

Contents

1	Introduction	1
1.1	Problem Description	1
1.2	Solution Overview	1
2	Implementation	1
2.1	GraphFrames	1
2.2	Label Propagation Algorithm (LPA)	2
2.2.1	Custom Implementation using PySpark	2
2.2.2	GraphFrames Implementation	3
2.2.3	Comparison Between the Two Implementations	3
2.3	GraphFrames Community Detection Framework	4
2.3.1	Louvain	4
2.3.2	FastGreedy	4
2.3.3	K-Clique Percolation	4
3	Experimental Evaluation	5
3.1	Dataset Analysis	5
3.1.1	Email-Eu-core network	5
3.1.2	Amazon product co-purchasing network	5
3.2	Dataset Preprocessing	6
3.3	Results	7
3.3.1	Email-Eu-core network	7
3.3.2	Amazon product co-purchasing network	9
4	Conclusion	11

1 Introduction

1.1 Problem Description

Graph community detection identifies groups of nodes that are more closely connected to one another than to the rest of the graph. These groups, known as *communities*, are characterized by high internal density and low inter-group connectivity. By organizing nodes into communities, community detection provides a compact representation of graph topology, replacing large numbers of individual connections with high-level functional units.

Community detection has applications across several domains, including social and behavioral analysis, e-commerce, and bioinformatics. In social and behavioral analysis, it can identify interest groups and informal organizational hierarchies. In e-commerce, it can reveal natural product bundles and cohesive consumer segments based on co-purchase behavior. In bioinformatics, it can uncover functional protein modules, metabolic pathways, and gene interaction clusters.

The main limitations of graph community detection are algorithmic complexity, memory requirements, and hardware constraints. Classical graph partitioning problems are NP-hard, while some algorithms have high polynomial time complexity, such as $O(V^3)$ or $O(E^2)$. Even heuristic methods, such as Louvain and Walktrap, can struggle when graphs grow to millions of vertices (V) and edges (E). Moreover, large real-world graphs can exceed the memory capacity of a single server.

1.2 Solution Overview

To address these scalability limitations, graph storage and computation can be distributed across a cluster instead of being confined to a single server. Apache Spark provides a distributed computing framework for efficient, in-memory processing of large datasets. GraphFrames extends Spark DataFrames to graph analytics by representing vertices and edges as distributed tables. This representation allows graph algorithms such as Label Propagation to run in parallel, reducing the memory and computational load on each machine.

In this project, we evaluate distributed implementations of three established community detection algorithms—K-Cliques, Louvain, and FastGreedy—using Apache Spark GraphFrames and the framework proposed by Apostol et al. [1]. The Label Propagation Algorithm (LPA) [2] serves as a baseline. We compare a custom parallel LPA implementation written in PySpark with the implementation provided by GraphFrames¹. The experiments use two real-world networks of different sizes with ground-truth community labels: the email-Eu-core network [3] and the Amazon product co-purchasing network [4]. We assess scalability by measuring execution time with different numbers of CPU cores. We evaluate community quality by comparing the detected communities with the ground-truth labels. The goal is to determine whether these algorithms scale effectively and remain suitable for large real-world graphs.

2 Implementation

2.1 GraphFrames

GraphFrames is a package for large-scale graph processing and analytics. Built on Apache Spark and its DataFrame abstraction, it provides built-in, easy-to-use distributed graph algorithms.

GraphFrames represents a graph with two DataFrames: one for vertices and one for edges.

¹GraphFrames LPA documentation, <https://graphframes.io/04-user-guide/06-graph-clustering.html>.

2.2 Label Propagation Algorithm (LPA)

The Label Propagation Algorithm (LPA) is a simple community detection method that iteratively assigns each node the most common label among its immediate neighbors. The main execution loop is as follows:

1. Initialize the label of each node. For every node $x \in V$, set

$$C_x(0) = x.$$

2. Set the iteration counter to $t = 1$.
3. Arrange the nodes in a random order and denote the resulting sequence by X .
4. For each node $x \in X$, processed in the selected order, let $x_{i_1}, \dots, x_{i_k}$ be its neighbors. If the first m neighbors have already been updated during iteration t , update the label of x as

$$C_x(t) = f\left(C_{x_{i_1}}(t), \dots, C_{x_{i_m}}(t), C_{x_{i_{m+1}}}(t-1), \dots, C_{x_{i_k}}(t-1)\right),$$

where f returns the most frequent label among the neighbors. If several labels have the same maximum frequency, select one of them at random.

5. Stop when every node has one of the most frequent labels among its neighbors. Otherwise, set $t \leftarrow t + 1$ and return to Step 3.

We evaluate both a custom parallel implementation written in PySpark and the existing implementation provided by GraphFrames.

2.2.1 Custom Implementation using PySpark

Our LPA implementation uses PySpark Resilient Distributed Datasets (RDDs). It has three main stages: graph preparation, distributed label propagation, and convergence management.

Graph Loading and Deterministic Re-indexing. The graph is loaded from an edge-list file with `sc.textFile()`. Comment lines, empty lines, and self-loops are removed. Because the graph is treated as undirected, each edge (u, v) is converted to the canonical pair $(\min(u, v), \max(u, v))$. Duplicate edges are combined with `reduceByKey()`, and the number of occurrences defines the edge weight.

The original node identifiers are remapped to the contiguous range $0, \dots, N - 1$ as follows:

1. Distinct node identifiers are extracted and sorted with `sortBy()` to produce a deterministic order.
2. `zipWithIndex()` assigns each node a contiguous identifier and creates the cached `node_map` RDD.
3. Two distributed `join()` operations with `node_map` replace the source and destination identifiers of each edge.
4. Each undirected edge is expanded into the two directed records `(src, (dst, weight))` and `(dst, (src, weight))`. The resulting edge RDD is cached.

Each node initially receives its remapped identifier as its community label. The edge and label RDDs are materialized with `count()` before execution time is measured. Ground-truth communities are remapped with the same `node_map` and stored for later evaluation.

Distributed Propagation Mechanics (lpa_step_robust). At each iteration, the algorithm updates labels through the following distributed operations:

- **Message generation and aggregation:** `edges.join(labels)` associates each directed edge with the current label of its source node. The source label contributes the edge weight as a vote for the destination node. Votes for the same destination and label are summed with `reduceByKey()`.
- **State association:** The aggregated votes are grouped by destination node with `groupByKey()` and joined with the node’s current label.
- **Parity-based updates:** To reduce label oscillation, only one parity class is updated per iteration. A node is eligible for an update when

$$\text{node_id} \bmod 2 = \text{iteration} \bmod 2. \quad (1)$$

Nodes in the other parity class retain their current label.

- **Deterministic tie-breaking:** An eligible node selects the label with the highest total vote weight. If its current label is among the best candidates, it is retained. Otherwise, the candidate with the smallest numerical identifier is selected.

Lineage Management and Convergence (run_robust_lpa). The propagation step continues until no labels change or the maximum number of iterations is reached. The implementation manages the growing RDD lineage as follows:

- The result of each propagation step is checkpointed to truncate its dependency lineage.
- A `sum()` action computes the number of changed labels (`changed_count`), materializes the step result, and determines whether the algorithm has converged.
- The updated label RDD is cached and checkpointed before it is used in the next iteration.
- Label RDDs from completed iterations are removed from memory with `unpersist()`.

2.2.2 GraphFrames Implementation

In this implementation, GraphFrames receives vertices and edges as distributed DataFrames and runs LPA through its default GraphX backend². Each vertex initially receives its own identifier as a label. During each of the ten synchronous iterations, labels are propagated to neighboring vertices, and each vertex adopts the most frequent received label³. GraphX runs the iterative computation inside the JVM, so the per-vertex update loop does not execute in Python. The resulting vertex DataFrame, which contains the final labels, is cached for later evaluation.

2.2.3 Comparison Between the Two Implementations

Although both implementations follow the same label propagation principle, they use different execution models and update strategies.

²GraphFrames Python API source code, <https://github.com/graphframes/graphframes/blob/main/python/graphframes/graphframe.py>.

³Apache Spark GraphX LabelPropagation source code, <https://github.com/apache/spark/blob/master/graphx/src/main/scala/org/apache/spark/graphx/lib/LabelPropagation.scala>.

Execution Model. The custom implementation uses PySpark RDD transformations, and its update functions run in Python workers. This requires data serialization between the JVM and Python during the iterative computation. The GraphFrames implementation starts from DataFrames but delegates LPA to the GraphX backend, where the propagation loop runs in Scala inside the JVM³. It therefore avoids per-vertex Python execution, although the iterative algorithm uses GraphX RDDs rather than Spark SQL operations.

Label Updates. The custom implementation uses weighted votes, so duplicate edges increase the influence of a neighboring label. It updates one parity class of vertices per iteration and applies deterministic tie-breaking: the current label is retained when possible; otherwise, the smallest candidate label is selected. GraphX performs synchronous, unweighted updates for all vertices, propagates labels in both directions along each edge, and selects the most frequent label. Its implementation does not define an explicit secondary rule for ties³.

2.3 GraphFrames Community Detection Framework

We evaluate the distributed community detection framework proposed by Apostol et al. [1]. The framework implements modularity-based and clique-percolation algorithms with Spark GraphFrames and the DataFrame API.

2.3.1 Louvain

The Louvain algorithm optimizes graph modularity Q through a two-phase iterative process [1]. During the local moving phase (S_1), each node evaluates the modularity gain ΔQ from moving to a neighboring community. It greedily joins the community that produces the largest positive gain until the algorithm reaches a local optimum. During the aggregation phase (S_2), communities are contracted into super-nodes to form a smaller meta-graph, where weighted edges represent aggregated inter-community connections. The implementation maps these steps to Spark DataFrames and persists intermediate graph states across iterations. In our benchmark, the resolution parameter is set to $\gamma = 1$.

2.3.2 FastGreedy

FastGreedy follows the Clauset–Newman–Moore hierarchical agglomerative heuristic to optimize modularity [1]. It initializes each node as a singleton community and maintains the modularity changes ΔQ_{ij} associated with possible pairwise merges. At each step, the algorithm extracts the largest entry from a max-heap, merges the corresponding communities, and updates the gain matrix ΔQ for neighboring communities. This process continues until a single global partition remains. In the GraphFrames implementation, pairwise candidates are evaluated through relational DataFrame joins.

2.3.3 K-Clique Percolation

The k -Clique method identifies overlapping communities through clique adjacency [1]. A community is the maximal union of k -cliques, or complete subgraphs of size k , connected through a sequence of adjacent cliques that share $k - 1$ vertices.

The framework discovers k -cliques through structural motif queries submitted with the GraphFrames `find()` API. The `get_pattern_motif` function defines the required subgraph pattern. In our evaluation, we test clique sizes $k = 3$ and $k = 4$.

3 Experimental Evaluation

3.1 Dataset Analysis

We use two datasets from the Stanford Large Network Dataset Collection: the email-Eu-core network [3] and the Amazon product co-purchasing network [4]. Both datasets include ground-truth community labels.

3.1.1 Email-Eu-core network

The email-Eu-core dataset represents email exchanges between members of a large European research institution [3]. In the original directed graph, an edge (u, v) indicates that member u sent at least one email to member v . Communication with people outside the institution is excluded. Each node belongs to one of 42 departments, which serve as the ground-truth communities. Table 1 summarizes the original graph before preprocessing.

Metric	Value
Nodes	1,005
Edges	25,571
Ground-truth communities	42
Nodes in largest WCC	986 (0.981)
Edges in largest WCC	25,552 (0.999)
Nodes in largest SCC	803 (0.799)
Edges in largest SCC	24,729 (0.967)
Average clustering coefficient	0.3994
Number of triangles	105,461
Fraction of closed triangles	0.1085
Diameter (longest shortest path)	7
90th-percentile effective diameter	2.9

Table 1: Statistics of the original email-Eu-core network [3]. WCC and SCC denote weakly and strongly connected components, respectively. Values in parentheses are fractions of the total number of nodes or edges.

3.1.2 Amazon product co-purchasing network

The Amazon dataset is an undirected product co-purchasing network built from the website’s “Customers Who Bought This Item Also Bought” feature [4]. Nodes represent products, and edges connect products that are frequently purchased together. The ground-truth communities are derived from product categories: each connected component within a category forms a separate community, while communities with fewer than three nodes are excluded. Products may belong to multiple categories, so these communities can overlap. SNAP provides both the complete set of deduplicated communities and a subset of the 5,000 highest-quality communities. We use the latter for evaluation. Table 2 summarizes the original graph before preprocessing.

Metric	Value
Nodes	334,863
Edges	925,872
Ground-truth communities (used for evaluation)	5,000
Nodes in largest WCC	334,863 (1.000)
Edges in largest WCC	925,872 (1.000)
Nodes in largest SCC	334,863 (1.000)
Edges in largest SCC	925,872 (1.000)
Average clustering coefficient	0.3967
Number of triangles	667,129
Fraction of closed triangles	0.07925
Diameter (longest shortest path)	44
90th-percentile effective diameter	15

Table 2: Statistics of the original Amazon product co-purchasing network [4]. WCC and SCC denote weakly and strongly connected components, respectively; they coincide for this undirected graph. Values in parentheses are fractions of the total number of nodes or edges.

3.2 Dataset Preprocessing

Both datasets use the same preprocessing pipeline. The only difference is the final representation required by each implementation: the GraphFrames algorithms use vertex and edge DataFrames, while the custom LPA uses RDDs directly.

Each edge-list file is first loaded with `sc.textFile()`. Dataset-specific metadata is removed, each remaining line is split into two node identifiers, and self-loops are discarded.

Both graphs are treated as undirected. Each edge (u, v) is therefore converted to the canonical pair

$$(\min(u, v), \max(u, v)).$$

This representation merges opposite directions and repeated edges. Duplicate occurrences are aggregated with `reduceByKey()`, and their count becomes the edge weight.

The original node identifiers may be sparse and non-contiguous. To create a compact and deterministic representation, all distinct identifiers are extracted, sorted, and assigned a new index in the range $0, \dots, N - 1$ with `zipWithIndex()`. The resulting `node_map` RDD contains pairs of the form `(original_id, new_id)`. Both endpoints of each edge are then remapped through two distributed joins with `node_map`. This keeps the mapping distributed and avoids collecting it on the driver.

After these shared steps, the data is converted into the format required by each implementation:

- **GraphFrames algorithms:** Each undirected edge is expanded into two directed rows, `(src, dst, relationship, weight)` and `(dst, src, relationship, weight)`. The relationship value is set to `friend` for compatibility with the evaluated implementations. The vertex DataFrame contains the columns `name`, `id`, and `community`. Together, the vertex and edge DataFrames define the input **GraphFrame**.
- **Custom LPA:** Each undirected edge is expanded into two directed key-value records, `(src, (dst, weight))` and `(dst, (src, weight))`. This adjacency RDD supports the joins and weighted message aggregation used during label propagation. Each node initially receives its remapped identifier as its label.

The final graph structures are cached and materialized with `count()` before timing begins. Therefore, the reported execution times exclude file loading, node re-indexing, and graph construction.

3.3 Results

We evaluate the scalability of each algorithm using several CPU-core configurations. The experiments were run in Spark local mode on a university-cluster node. Each dataset used a fixed driver-memory allocation (`--driver-memory xg`), while the number of worker threads varied through `--master local[x]`.

For both datasets, we report the number of detected communities, their median and maximum sizes, and the percentage of singleton communities. These statistics describe the granularity and balance of each output. We also report the number of nodes evaluated against the ground truth (GT). The quality metrics used for each dataset are described in the corresponding section.

3.3.1 Email-Eu-core network

The email-Eu-core experiments used a fixed 8 GB of RAM and 2, 4, or 8 CPU cores. For Louvain, we fixed the resolution parameter at $\gamma = 1$.

For the k -Clique method, we set the clique size to $k = 3$. We also tested $k = 4$, but it consistently caused Out-of-Memory (OOM) failures. The problem arises from the `get_pattern_motif` implementation, which defines bidirectional edge clauses for every node pair without enforcing a node order. For $k = 4$, the motif becomes a chain of 12 relational self-joins, and Spark materializes all $4! = 24$ permutations of each geometric 4-clique. The resulting combinatorial growth in intermediate states quickly exhausts the allocated 8 GB of driver memory when `collect()` triggers execution.

Algorithm	2 cores	4 cores	8 cores
LPA (ours)	2524.11	1259.45	632.76
LPA (GraphFrames)	3.42	3.06	3.36
Louvain	871.89	839.39	771.79
3-Cliques	282.54	282.12	281.57
FastGreedy	187.20	184.29	174.65

Table 3: Execution times in seconds on the email-Eu-core network using 2, 4, and 8 CPU cores.

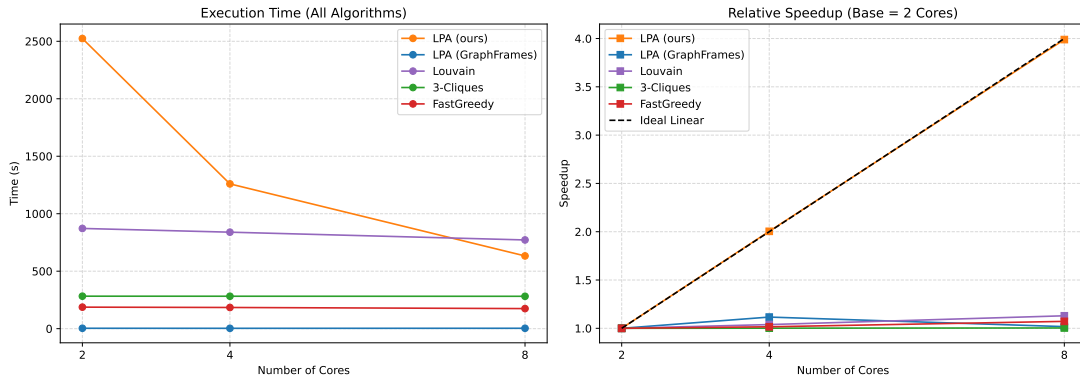


Figure 1: Execution times and relative speedup on the email-Eu-core network. Speedup is measured relative to the 2-core configuration; the reference line indicates ideal linear speedup.

Table 3 and Figure 1 show that our LPA is the only tested implementation with approximately linear scaling. Its execution time nearly halves whenever the number of cores doubles, producing a speedup of about $4\times$ from 2 to 8 cores. The other algorithms show little or no improvement. Nevertheless, LPA (GraphFrames) remains at least two orders of magnitude faster than our implementation in every configuration.

Clustering Quality. We evaluate clustering quality against the 42 disjoint ground-truth departments using Normalized Mutual Information (NMI) and the Adjusted Rand Index (ARI), as reported in Table 4. Both metrics compare partitions independently of their numerical labels. NMI measures shared information on a $[0, 1]$ scale without adjusting for chance. ARI measures pairwise clustering agreement after correcting for chance, with values near zero indicating chance-level agreement.

Algorithm	NMI	ARI	Num. comms.	Median size	Max size	Singletons (%)	Evaluated nodes (GT)
LPA (ours)	0.2433	0.0286	4	81	806	0.0	986
LPA (GraphFrames)	0.0869	0.0008	4	27	930	25.0	986
Louvain	0.6980	0.5241	32	28	108	0.0	986
3-Cliques	0.0000	0.0000	1	875	875	0.0	875
FastGreedy	0.5791	0.3436	21	16	197	0.0	986

Table 4: Clustering quality and community statistics on the email-Eu-core network. GT denotes ground truth; the number of evaluated nodes is reported separately for each algorithm.

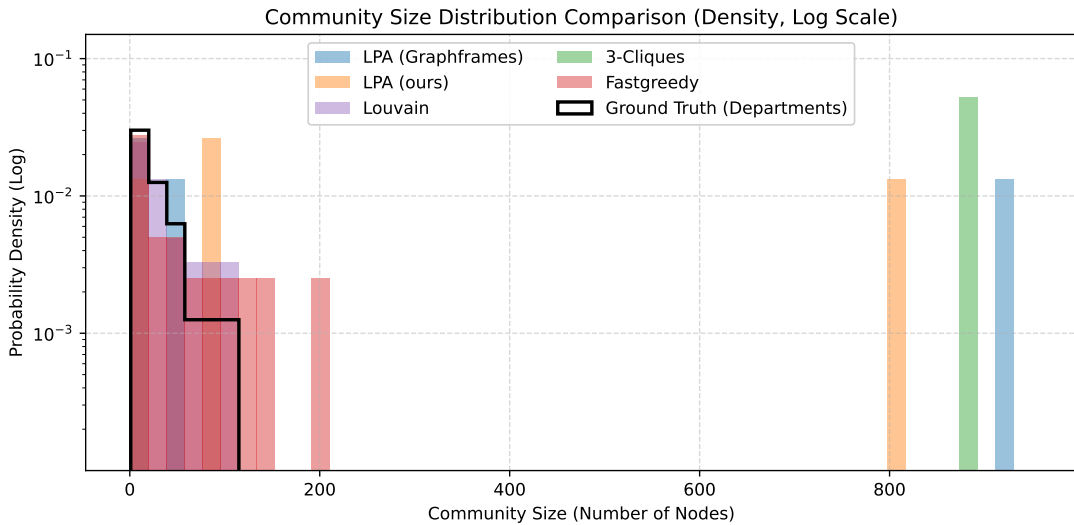


Figure 2: Community size distributions on the email-Eu-core network for LPA (GraphFrames), LPA (ours), Louvain, and the ground-truth departments. Probability density is shown on a logarithmic scale.

Table 4 and Figure 2 show that Louvain achieves the highest clustering quality. It closely matches the reference structure, identifying 32 communities with a maximum size of 108 nodes.

In contrast, both LPA implementations and the 3-Cliques method show severe community degeneration. Rapid label absorption across the dense core reduces each LPA output to only four communities, dominated by a cluster of 806 or 930 nodes. The near-zero ARI scores reflect this collapse. Figure 2 shows a clear density peak near 800–900 nodes, while the ground-truth

departments contain fewer than 200 nodes. Similarly, 3-Cliques merges almost all adjacent triangles into one community of 875 nodes, resulting in NMI and ARI scores of zero.

3.3.2 Amazon product co-purchasing network

The Amazon experiments used a fixed 16 GB of RAM and 4, 8, 16, or 20 CPU cores. Under these conditions, only the two LPA implementations completed. The k -Clique method with $k = 3$ caused an OOM failure for the reason discussed in Section 3.3.1. Louvain and FastGreedy both use the `get_nodes_degrees` routine, which was projected to require at least 20 hours for degree initialization alone. This cost comes from calling Spark’s `collect()` once for every node inside a sequential Python loop.

Algorithm	4 cores	8 cores	16 cores	20 cores
LPA (ours)	1538.13	774.72	384.63	338.19
LPA (GraphFrames)	37.05	32.35	30.70	32.34

Table 5: Execution times in seconds on the Amazon product co-purchasing network using 4, 8, 16, and 20 CPU cores.

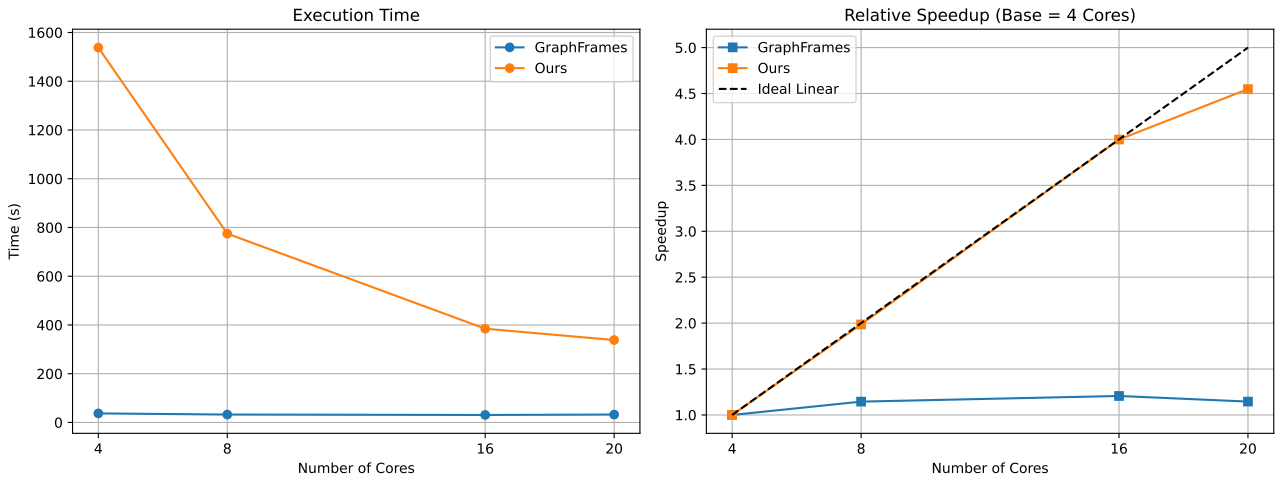


Figure 3: Execution times and relative speedup on the Amazon product co-purchasing network for LPA (GraphFrames) and LPA (ours). Speedup is measured relative to the 4-core configuration; the reference line indicates ideal linear speedup.

Table 5 and Figure 3 show that our LPA implementation scales nearly linearly from 4 to 16 cores and becomes slightly sub-linear at 20 cores. LPA (GraphFrames) remains more than an order of magnitude faster in every configuration. However, its speedup plateaus between 4 and 16 cores and decreases slightly at 20 cores, where task-scheduling and synchronization costs begin to dominate its short runtime.

Clustering Quality. We evaluate Amazon community quality against the partial and overlapping set of 5,000 ground-truth communities using the Average F_1 score (Avg F_1), as reported in Table 6. Unlike global partition metrics, Avg F_1 matches each ground-truth community with its best predicted cluster. It therefore provides a suitable harmonic mean of precision and recall for incomplete, overlapping community structures [4].

Algorithm	Avg F_1	Num. comms.	Median size	Max size	Singletons (%)	Evaluated nodes (GT)
LPA (ours)	0.9033	35,446	6	440	1.60	16,716
LPA (GraphFrames)	0.8555	27,049	7	627	5.83	16,716

Table 6: Clustering quality and community statistics on the Amazon product co-purchasing network, evaluated against the top 5,000 ground-truth communities. GT denotes ground truth. NMI and ARI are omitted for Amazon due to the overlapping and partial nature of the top-5000 ground-truth communities.

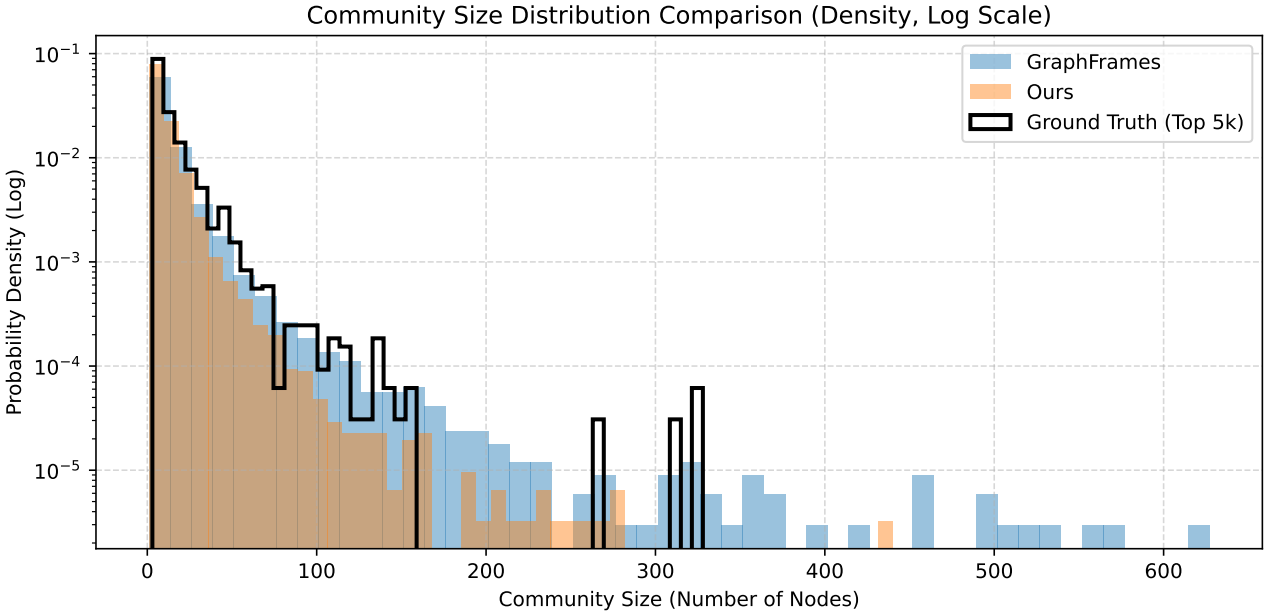


Figure 4: Community size distributions on the Amazon product co-purchasing network for LPA (GraphFrames), LPA (ours), and the top 5,000 ground-truth communities. Probability density is shown on a logarithmic scale.

Table 6 and Figure 4 show that both LPA implementations recover the product communities well. They achieve high Avg F_1 scores without the large-cluster collapse observed on the email network.

Our RDD-based implementation aligns more closely with the ground truth than GraphFrames. It produces a finer partition, with more communities, a smaller maximum community size (440 versus 627 nodes), and fewer singletons.

As shown in Figure 4, both community-size distributions follow the heavy-tailed decay of the ground truth. However, GraphFrames produces larger tail communities of more than 500 nodes. Our implementation avoids these oversized communities and better preserves local community boundaries.

4 Conclusion

In this project, we evaluated several distributed community detection implementations on real-world graphs. We measured execution time with different numbers of CPU cores and compared the detected communities with ground-truth labels. Our goal was to determine whether the algorithms could scale to large graphs while making effective use of additional computational resources.

The main findings are as follows:

- The GraphFrames LPA implementation provided the best overall balance between execution time and community detection quality. It was by far the fastest method on both datasets while maintaining competitive agreement with the ground-truth communities. This performance highlights the efficiency of its JVM-based GraphX backend, which performs iterative message passing without per-vertex Python execution and manages distributed graph state internally. Increasing the number of cores did not reduce its execution time. Because the algorithm was already fast, the limited graph-level parallelism could not offset Spark’s fixed scheduling and synchronization costs. In addition, Spark ran in local mode on a single university-cluster node, so the additional worker threads shared the same memory and JVM resources.
- Our custom LPA implementation showed the strongest scaling across the tested configurations. Its execution time was reduced by approximately half whenever the number of cores doubled, showing that its partitioned RDD transformations used the additional parallelism effectively. However, it remained considerably slower than GraphFrames LPA because of Python execution overhead and the repeated joins, aggregations, and materialization steps required at each iteration.
- The evaluated K-Cliques, FastGreedy, and Louvain implementations scaled poorly as graph size increased. On email-Eu-core, additional cores produced little improvement. On the larger Amazon network, none of the three methods completed within practical time and memory limits. K-Cliques exhausted the available memory because its motif construction generates many self-joins and redundant clique permutations, producing a combinatorial number of intermediate results before collecting them on the driver. Louvain and FastGreedy were limited by their shared `get_nodes_degrees` procedure, whose degree computation became prohibitively expensive on the larger graph. These outcomes are not limitations of GraphFrames itself, as its LPA implementation processed both datasets efficiently. Instead, they reveal bottlenecks in the proposed implementations, including redundant enumeration, expensive intermediate materialization, driver-side collection, and preprocessing steps that do not scale with graph size.

References

- [1] Elena-Simona Apostol, Adrian-Cosmin Cojocaru, and Ciprian-Octavian Truică. “Large-Scale Graphs Community Detection Using Spark GraphFrames”. In: *2024 23rd International Symposium on Parallel and Distributed Computing (ISPDC)*. IEEE, 2024, pp. 1–5. DOI: 10.1109/ISPDC62236.2024.10705389.
- [2] Usha Nandini Raghavan, Réka Albert, and Soundar Kumara. “Near Linear Time Algorithm to Detect Community Structures in Large-Scale Networks”. In: *Physical Review E* 76.3 (2007), p. 036106. DOI: 10.1103/PhysRevE.76.036106.
- [3] Hao Yin et al. “Local Higher-Order Graph Clustering”. In: *KDD : proceedings. International Conference on Knowledge Discovery & Data Mining*. Vol. 2017. Aug. 2017, pp. 555–564. DOI: 10.1145/3097983.3098069.
- [4] Jaewon Yang and Jure Leskovec. “Defining and evaluating network communities based on ground-truth”. In: *Knowledge and Information Systems* 42 (Jan. 2015), pp. 181–213. DOI: 10.1007/s10115-013-0693-z.